

We want to test the LLM summarization method first -> then pipeline from mongo -> then the summarization_router api endpoints

We also want to test concurrency, edge cases, parameter variations, benchmarking, error handling, etc

Test 1: Basic Functionality

Test 2: Mock-Based Unit Tests

Test 3: Concurrency Behavior

Test 4: Edge Cases

Test 5: Parameter Variations

Test 6: Benchmarking

Test 7: Error Handling

```
In [23]: import sys
import os

# Add the backend directory to Python path so we can import from app
backend_path = os.path.abspath('../..') # Go up 3 levels from current note
if backend_path not in sys.path:
    sys.path.append(backend_path)

print(f"Added to Python path: {backend_path}")
print(f"Current working directory: {os.getcwd()}")
```

Added to Python path: /Users/pranavkomarla/Desktop/MLOPS/LLM Project/backend
 Current working directory: /Users/pranavkomarla/Desktop/MLOPS/LLM Project/backend/app/testing

```
In [24]: # Test setup and imports
import pytest
from unittest.mock import Mock, AsyncMock, patch
import asyncio
from typing import List
import os
from dotenv import load_dotenv

# Load environment variables
load_dotenv()

# Import the function to test
from app.domain.llm_summarization.services.llm_routines import summarize_article
from app.core.config import config
```

```
In [25]: # Test data setup
SAMPLE_ARTICLE_CONTENT = """
Artificial Intelligence (AI) has revolutionized the technology industry in recent years.
Companies like OpenAI, Google, and Microsoft have invested billions of dollars in AI research.
The introduction of large language models such as GPT-4 has transformed how we interact with technology.

Machine learning algorithms are now being used in healthcare to diagnose diseases.
A recent study published in Nature Medicine showed that AI systems achieved 92% accuracy in diagnosing skin cancer,
compared to 87% accuracy for human radiologists.

The financial sector has also embraced AI for algorithmic trading and risk management.
JPMorgan Chase reported a 15% increase in trading profits after implementing AI-driven strategies.
However, concerns about job displacement and ethical implications continue to persist.

Regulatory bodies worldwide are working to establish guidelines for AI development.
The European Union's AI Act, passed in 2024, sets strict requirements for high-risk AI applications.
Similar legislation is being considered in the United States and other countries.

Despite the challenges, the AI industry is expected to grow from $100 billion in 2023 to $1.1 trillion by 2030,
according to a report by McKinsey Global Institute. This growth is driven by increasing investment in AI research
and continued technological advancements.
"""

SAMPLE_TITLE = "AI Revolution: Transforming Industries and Society"
SAMPLE_URL = "https://example.com/ai-revolution-article"
```

```
In [26]: from langchain_openai import ChatOpenAI
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Test 1: Basic functionality test with real LLM (requires OpenAI API key)
async def test_summarize_article_text_basic():
    """Test the basic functionality of summarize_article_text with real LLM"""

    # Check if OpenAI API key is available
    if not os.getenv("OPENAI_API_KEY"):
```

```

        print("Skipping real LLM test - OPENAI_API_KEY not found")
        return

    # Initialize real components
    llm = ChatOpenAI(
        model=config.OPENAI_MODEL,
        api_key=config.OPENAI_API_KEY,
        temperature=0.1
    )

    splitter = RecursiveCharacterTextSplitter(
        chunk_size=config.SUMMARY_MAP_CHUNK_SIZE,
        chunk_overlap=config.SUMMARY_MAP_CHUNK_OVERLAP
    )

    # Test parameters
    n_map_bullets = 3

    print("Testing summarize_article_text with real LLM...")
    print(f"Article length: {len(SAMPLE_ARTICLE_CONTENT)} characters")
    print(f"Chunk size: {config.SUMMARY_MAP_CHUNK_SIZE}")
    print(f"Bullets per chunk: {n_map_bullets}")

    try:
        result = await summarize_article_text(
            llm=llm,
            splitter=splitter,
            title=SAMPLE_TITLE,
            url=SAMPLE_URL,
            content=SAMPLE_ARTICLE_CONTENT,
            n_map_bullets=n_map_bullets
        )

        print("Test passed!")
        print("Generated summary:")
        print("-" * 50)
        print(result)
        print("-" * 50)

        # Basic assertions
        assert isinstance(result, str), "Result should be a string"
        assert len(result) > 0, "Result should not be empty"
        assert "Key Points" in result or "Takeaway" in result, "Result should contain key points or takeaway"

        return result

    except Exception as e:
        print(f"Test failed with error: {e}")
        raise

# Run the basic test
result = await test_summarize_article_text_basic()

```

Testing summarize_article_text with real LLM...

Article length: 1437 characters

Chunk size: 1000

Bullets per chunk: 3

Test passed!

Generated summary:

– Key Points

– Major companies like OpenAI, Google, and Microsoft have invested billions in AI.

– AI systems have achieved 94% accuracy in early-stage cancer detection, outperforming human radiologists.

– JPMorgan Chase experienced a 15% increase in trading profits due to AI algorithms.

– The European Union's AI Act, enacted in 2024, imposes strict regulations on AI.

– The AI industry is projected to grow from \$100 billion in 2023 to over \$1.8 trillion by 2030, according to McKinsey.

– Takeaway

The AI revolution is significantly transforming industries, with substantial investments from leading tech companies and impressive advancements in accuracy and profitability. Regulatory frameworks like the EU's AI Act are emerging to manage this growth, which is expected to skyrocket in the coming years.

– Source: <https://example.com/ai-revolution-article>

In [27]: `from app.domain.llm_summarization.services.llm_prompts import map_prompt, re
from app.domain.llm_summarization.services.llm_routines import build_map_cha
from app.domain.llm_summarization.services.llm_routines import summarize_art`

Test 2: Mock-based unit tests (no API calls required)

Fixed Mock Tests with Fresh Mocks for Each Test

async def test_basic_functionality_fixed():

"""Test basic functionality with fresh mocks"""

from unittest.mock **import** Mock, patch

Create fresh mocks for this test

 mock_llm = Mock(spec=ChatOpenAI)

 mock_splitter = Mock(spec=RecursiveCharacterTextSplitter)

 mock_map_chain = Mock()

 mock_reduce_chain = Mock()

Mock the splitter to return chunks

 chunks = ["chunk1", "chunk2"]

 mock_splitter.split_text.return_value = chunks

Mock map chain to return same response for all chunks

 map_response = "• Test bullet point"

 mock_map_chain.invoke.return_value = map_response

Mock reduce chain response

 expected_summary = "Key Points:\n• Test bullet point\n• Test bullet point"

```

mock_reduce_chain.invoke.return_value = expected_summary

# Patch the chain builders
with patch('app.domain.llm_summarization.services.llm_routines.build_map', mock_build_map),
     patch('app.domain.llm_summarization.services.llm_routines.build_reduce_chain', mock_build_reduce):

    # Setup the patches
    mock_build_map.return_value = mock_map_chain
    mock_build_reduce.return_value = mock_reduce_chain

    # Test parameters
    title = "Test Article"
    url = "https://example.com"
    content = "Test content"
    n_map_bullets = 2

    # Call the function
    result = await summarize_article_text(
        llm=mock_llm,
        splitter=mock_splitter,
        title=title,
        url=url,
        content=content,
        n_map_bullets=n_map_bullets
    )

    # Basic assertions
    assert result == expected_summary.strip()
    mock_splitter.split_text.assert_called_once_with(content)
    mock_build_map.assert_called_once_with(mock_llm, n_bullets=n_map_bullets)
    mock_build_reduce.assert_called_once_with(mock_llm)

    # Verify map chain was called for each chunk
    assert mock_map_chain.invoke.call_count == len(chunks)

    # Verify reduce chain was called
    mock_reduce_chain.invoke.assert_called_once()
    call_args = mock_reduce_chain.invoke.call_args[0][0]
    assert call_args["title"] == title
    assert call_args["url"] == url
    assert "bullets" in call_args

    print("Basic functionality test passed!")

async def test_none_values_fixed():
    """Test handling of None title and URL with fresh mocks"""

    from unittest.mock import Mock, patch

    # Create fresh mocks for this test
    mock_llm = Mock(spec=ChatOpenAI)
    mock_splitter = Mock(spec=RecursiveCharacterTextSplitter)
    mock_map_chain = Mock()
    mock_reduce_chain = Mock()

    mock_splitter.split_text.return_value = ["chunk1"]

```

```

mock_map_chain.invoke.return_value = "• Test point"
mock_reduce_chain.invoke.return_value = "Test summary"

with patch('app.domain.llm_summarization.services.llm_routines.build_map') as mock_build_map,
    patch('app.domain.llm_summarization.services.llm_routines.build_reduce') as mock_build_reduce:

    mock_build_map.return_value = mock_map_chain
    mock_build_reduce.return_value = mock_reduce_chain

    result = await summarize_article_text(
        llm=mock_llm,
        splitter=mock_splitter,
        title=None,
        url=None,
        content="test content",
        n_map_bullets=1
    )

    # Verify reduce chain was called with default values
    mock_reduce_chain.invoke.assert_called_once()
    call_args = mock_reduce_chain.invoke.call_args[0][0]
    assert call_args["title"] == "(untitled)"
    assert call_args["url"] == ""

    print("None values test passed!")

async def test_empty_content_fixed():
    """Test handling of empty content with fresh mocks"""

    from unittest.mock import Mock, patch

    # Create fresh mocks for this test
    mock_llm = Mock(spec=ChatOpenAI)
    mock_splitter = Mock(spec=RecursiveCharacterTextSplitter)
    mock_map_chain = Mock()
    mock_reduce_chain = Mock()

    mock_splitter.split_text.return_value = [] # Empty chunks
    mock_reduce_chain.invoke.return_value = "Empty summary"

    with patch('app.domain.llm_summarization.services.llm_routines.build_map') as mock_build_map,
        patch('app.domain.llm_summarization.services.llm_routines.build_reduce') as mock_build_reduce:

        mock_build_map.return_value = mock_map_chain
        mock_build_reduce.return_value = mock_reduce_chain

        result = await summarize_article_text(
            llm=mock_llm,
            splitter=mock_splitter,
            title="Empty Article",
            url="https://example.com",
            content="",
            n_map_bullets=2
        )

        # Verify reduce chain was called with empty bullets

```

```

        mock_reduce_chain.invoke.assert_called_once()
        call_args = mock_reduce_chain.invoke.call_args[0][0]
        assert call_args["bullets"] == ""

        print("Empty content test passed!")

# Run all fixed tests
print("Running fixed mock tests...")
await test_basic_functionality_fixed()
await test_none_values_fixed()
await test_empty_content_fixed()
print("All fixed mock tests passed!")

```

Running fixed mock tests...
 Basic functionality test passed!
 None values test passed!
 Empty content test passed!
 All fixed mock tests passed!

```

In [28]: # Test 3: Performance and concurrency tests
async def test_concurrency_behavior():
    """Test the concurrency behavior with multiple chunks"""

    # Check if OpenAI API key is available
    if not os.getenv("OPENAI_API_KEY"):
        print("⚠️ Skipping concurrency test - OPENAI_API_KEY not found")
        return

    llm = ChatOpenAI(
        model=config.OPENAI_MODEL,
        api_key=config.OPENAI_API_KEY,
        temperature=0.1
    )

    # Create a splitter that will generate many chunks
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=200, # Smaller chunks to create more of them
        chunk_overlap=50
    )

    # Create longer content to ensure multiple chunks
    long_content = SAMPLE_ARTICLE_CONTENT * 3 # Triple the content

    print("Testing concurrency behavior...")
    print(f"Content length: {len(long_content)} characters")
    print(f"Max concurrency: {config.MAX_MAP_CONCURRENCY}")

    import time
    start_time = time.time()

    try:
        result = await summarize_article_text(
            llm=llm,
            splitter=splitter,
            title="Concurrency Test Article",
            url="https://example.com/concurrency-test",

```

```

        content=long_content,
        n_map_bullets=2
    )

    end_time = time.time()
    duration = end_time - start_time

    print(f"Concurrency test completed in {duration:.2f} seconds")
    print(f"Result length: {len(result)} characters")

    # Basic assertions
    assert isinstance(result, str), "Result should be a string"
    assert len(result) > 0, "Result should not be empty"

    return result, duration

except Exception as e:
    print(f"Concurrency test failed: {e}")
    raise

# Run concurrency test
concurrency_result = await test_concurrency_behavior()

```

Testing concurrency behavior...
 Content length: 4311 characters
 Max concurrency: 8
 Concurrency test completed in 11.43 seconds
 Result length: 1347 characters

```

In [29]: # Test 4: Edge cases and error handling
async def test_edge_cases():
    """Test various edge cases and error scenarios"""

    print("Testing edge cases...")

    # Test with very short content
    short_content = "AI is important."

    if os.getenv("OPENAI_API_KEY"):
        llm = ChatOpenAI(
            model=config.OPENAI_MODEL,
            api_key=config.OPENAI_API_KEY,
            temperature=0.1
        )

        splitter = RecursiveCharacterTextSplitter(
            chunk_size=config.SUMMARY_MAP_CHUNK_SIZE,
            chunk_overlap=config.SUMMARY_MAP_CHUNK_OVERLAP
        )

        try:
            result = await summarize_article_text(
                llm=llm,
                splitter=splitter,
                title="Short Article",
                url="https://example.com/short",
            )

```

```

        content=short_content,
        n_map_bullets=1
    )

    print("Short content test passed")
    print(f"Result: {result[:100]}...")

    except Exception as e:
        print(f"Short content test failed: {e}")

    # Test with special characters and unicode
    unicode_content = "AI技术正在改变世界。🚀 人工智能的未来充满可能性。"

    if os.getenv("OPENAI_API_KEY"):
        try:
            result = await summarize_article_text(
                llm=llm,
                splitter=splitter,
                title="Unicode Article",
                url="https://example.com/unicode",
                content=unicode_content,
                n_map_bullets=2
            )

            print("Unicode content test passed")
            print(f"Result: {result[:100]}...")

        except Exception as e:
            print(f"Unicode content test failed: {e}")

    # Test with very long title
    long_title = "This is a very long title that might cause issues with the"

    if os.getenv("OPENAI_API_KEY"):
        try:
            result = await summarize_article_text(
                llm=llm,
                splitter=splitter,
                title=long_title,
                url="https://example.com/long-title",
                content=SAMPLE_ARTICLE_CONTENT,
                n_map_bullets=3
            )

            print("Long title test passed")
            print(f"Result length: {len(result)} characters")

        except Exception as e:
            print(f"Long title test failed: {e}")

    # Run edge case tests
    await test_edge_cases()

```

Testing edge cases...

Short content test passed

Result: – Key Points

- AI is crucial for various applications.

– Takeaway

AI plays a significant role...

Unicode content test passed

Result: – Key Points

- AI technology is transforming the world.
- The future of artificial intelligence...

Long title test passed

Result length: 898 characters

```
In [30]: # Test 5: Integration test with different parameters
async def test_parameter_variations():
    """Test the function with different parameter combinations"""

    if not os.getenv("OPENAI_API_KEY"):
        print("Skipping parameter variation tests – OPENAI_API_KEY not found")
        return

    llm = ChatOpenAI(
        model=config.OPENAI_MODEL,
        api_key=config.OPENAI_API_KEY,
        temperature=0.1
    )

    splitter = RecursiveCharacterTextSplitter(
        chunk_size=config.SUMMARY_MAP_CHUNK_SIZE,
        chunk_overlap=config.SUMMARY_MAP_CHUNK_OVERLAP
    )

    print("Testing parameter variations...")

    # Test different bullet counts
    bullet_counts = [1, 2, 4, 6]

    for n_bullets in bullet_counts:
        try:
            print(f"Testing with {n_bullets} bullets per chunk...")

            result = await summarize_article_text(
                llm=llm,
                splitter=splitter,
                title=f"Test Article – {n_bullets} bullets",
                url=f"https://example.com/test-{n_bullets}",
                content=SAMPLE_ARTICLE_CONTENT,
                n_map_bullets=n_bullets
            )

            print(f"{n_bullets} bullets test passed")
            print(f"Result length: {len(result)} characters")

        except Exception as e:
            print(f"{n_bullets} bullets test failed: {e}")
```

```

# Test different chunk sizes
chunk_sizes = [500, 1000, 1500]

for chunk_size in chunk_sizes:
    try:
        print(f"Testing with chunk size {chunk_size}...")

        test_splitter = RecursiveCharacterTextSplitter(
            chunk_size=chunk_size,
            chunk_overlap=200
        )

        result = await summarize_article_text(
            llm=llm,
            splitter=test_splitter,
            title=f"Test Article - chunk size {chunk_size}",
            url=f"https://example.com/chunk-{chunk_size}",
            content=SAMPLE_ARTICLE_CONTENT,
            n_map_bullets=3
        )

        print(f"Chunk size {chunk_size} test passed")
        print(f"Result length: {len(result)} characters")

    except Exception as e:
        print(f"Chunk size {chunk_size} test failed: {e}")

# Run parameter variation tests
await test_parameter_variations()

```

```

Testing parameter variations...
Testing with 1 bullets per chunk...
1 bullets test passed
Result length: 460 characters
Testing with 2 bullets per chunk...
2 bullets test passed
Result length: 709 characters
Testing with 4 bullets per chunk...
4 bullets test passed
Result length: 886 characters
Testing with 6 bullets per chunk...
6 bullets test passed
Result length: 1193 characters
Testing with chunk size 500...
Chunk size 500 test passed
Result length: 1285 characters
Testing with chunk size 1000...
Chunk size 1000 test passed
Result length: 844 characters
Testing with chunk size 1500...
Chunk size 1500 test passed
Result length: 688 characters

```

```

In [31]: # Test 6: Benchmarking and performance analysis
         async def benchmark_summarization():

```

```

"""Benchmark the summarization performance"""

if not os.getenv("OPENAI_API_KEY"):
    print("Skipping benchmark tests - OPENAI_API_KEY not found")
    return

llm = ChatOpenAI(
    model=config.OPENAI_MODEL,
    api_key=config.OPENAI_API_KEY,
    temperature=0.1
)

print("Running benchmark tests...")

# Test with different content sizes
content_sizes = [
    ("Small", SAMPLE_ARTICLE_CONTENT[:500]),
    ("Medium", SAMPLE_ARTICLE_CONTENT),
    ("Large", SAMPLE_ARTICLE_CONTENT * 2),
    ("Extra Large", SAMPLE_ARTICLE_CONTENT * 4)
]

results = []

for size_name, content in content_sizes:
    print(f"Benchmarking {size_name} content ({len(content)} chars)...")

    splitter = RecursiveCharacterTextSplitter(
        chunk_size=config.SUMMARY_MAP_CHUNK_SIZE,
        chunk_overlap=config.SUMMARY_MAP_CHUNK_OVERLAP
    )

    import time
    start_time = time.time()

    try:
        result = await summarize_article_text(
            llm=llm,
            splitter=splitter,
            title=f"Benchmark {size_name} Article",
            url=f"https://example.com/benchmark-{size_name.lower()}",
            content=content,
            n_map_bullets=3
        )

        end_time = time.time()
        duration = end_time - start_time

        # Calculate chunks created
        chunks = splitter.split_text(content)

        benchmark_result = {
            "size": size_name,
            "content_length": len(content),
            "chunks_created": len(chunks),
            "duration_seconds": duration,

```

```

        "chars_per_second": len(content) / duration if duration > 0
        "result_length": len(result)
    }

    results.append(benchmark_result)

    print(f"{size_name}: {duration:.2f}s, {len(chunks)} chunks, {ber

except Exception as e:
    print(f"{size_name} benchmark failed: {e}")

# Print summary
print("\nBenchmark Summary:")
print("-" * 80)
print(f"{'Size':<12} {'Content':<8} {'Chunks':<7} {'Duration':<10} {'Spe
print("-" * 80)

for result in results:
    print(f"{'result['size']':<12} {'result['content_length']':<8} {'result['
          f"{'result['duration_seconds']':<10.2f} {'result['chars_per_secor

return results

# Run benchmark tests
benchmark_results = await benchmark_summarization()

```

Running benchmark tests...

Benchmarking Small content (500 chars)...

Small: 3.93s, 1 chunks, 127 chars/s

Benchmarking Medium content (1437 chars)...

Medium: 7.17s, 2 chunks, 200 chars/s

Benchmarking Large content (2874 chars)...

Large: 8.39s, 4 chunks, 343 chars/s

Benchmarking Extra Large content (5748 chars)...

Extra Large: 7.99s, 7 chunks, 720 chars/s

Benchmark Summary:

Size	Content	Chunks	Duration	Speed	Result
Small	500	1	3.93	127	573
Medium	1437	2	7.17	200	911
Large	2874	4	8.39	343	1004
Extra Large	5748	7	7.99	720	899

```

In [32]: # Test 7: Error handling and exception scenarios
async def test_error_scenarios():
    """Test error handling and exception scenarios"""

    print("🔧 Testing error scenarios...")

    # Test with invalid LLM (should fail gracefully)
    try:
        invalid_llm = None

```

```

splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_ove

result = await summarize_article_text(
    llm=invalid_llm,
    splitter=splitter,
    title="Error Test",
    url="https://example.com/error",
    content="test content",
    n_map_bullets=2
)

print("Should have failed with invalid LLM")

except Exception as e:
    print(f"Correctly handled invalid LLM: {type(e).__name__}")

# Test with invalid splitter
try:
    llm = Mock(spec=ChatOpenAI)
    invalid_splitter = None

    result = await summarize_article_text(
        llm=llm,
        splitter=invalid_splitter,
        title="Error Test",
        url="https://example.com/error",
        content="test content",
        n_map_bullets=2
    )

    print("Should have failed with invalid splitter")

except Exception as e:
    print(f"Correctly handled invalid splitter: {type(e).__name__}")

# Test with negative bullet count
try:
    llm = Mock(spec=ChatOpenAI)
    splitter = Mock(spec=RecursiveCharacterTextSplitter)
    splitter.split_text.return_value = ["chunk1"]

    result = await summarize_article_text(
        llm=llm,
        splitter=splitter,
        title="Error Test",
        url="https://example.com/error",
        content="test content",
        n_map_bullets=-1 # Invalid negative value
    )

    print("Should have failed with negative bullet count")

except Exception as e:
    print(f"Correctly handled negative bullet count: {type(e).__name__}")

print("Error scenario tests completed")

```

```
# Run error scenario tests
await test_error_scenarios()
```

✓ Testing error scenarios...
 Correctly handled invalid LLM: TypeError
 Correctly handled invalid splitter: AttributeError
 Correctly handled negative bullet count: TypeError
 Error scenario tests completed

```
In [33]: # Additional Simple Tests
async def test_none_values():
    """Test handling of None title and URL"""

    from unittest.mock import Mock, patch

    mock_llm = Mock(spec=ChatOpenAI)
    mock_splitter = Mock(spec=RecursiveCharacterTextSplitter)
    mock_map_chain = Mock()
    mock_reduce_chain = Mock()

    mock_splitter.split_text.return_value = ["chunk1"]
    mock_map_chain.invoke.return_value = "• Test point"
    mock_reduce_chain.invoke.return_value = "Test summary"

    with patch('app.domain.llm_summarization.services.llm_routines.build_map', mock_map_chain), \
         patch('app.domain.llm_summarization.services.llm_routines.build_reduce_chain', mock_reduce_chain):

        mock_build_map.return_value = mock_map_chain
        mock_build_reduce.return_value = mock_reduce_chain

        result = await summarize_article_text(
            llm=mock_llm,
            splitter=mock_splitter,
            title=None,
            url=None,
            content="test content",
            n_map_bullets=1
        )

        # Verify reduce chain was called with default values
        mock_reduce_chain.invoke.assert_called_once()
        call_args = mock_reduce_chain.invoke.call_args[0][0]
        assert call_args["title"] == "(untitled)"
        assert call_args["url"] == ""

        print("None values test passed!")

    async def test_empty_content():
        """Test handling of empty content"""

        from unittest.mock import Mock, patch

        mock_llm = Mock(spec=ChatOpenAI)
        mock_splitter = Mock(spec=RecursiveCharacterTextSplitter)
        mock_map_chain = Mock()
```

```

mock_reduce_chain = Mock()

mock_splitter.split_text.return_value = [] # Empty chunks
mock_reduce_chain.invoke.return_value = "Empty summary"

with patch('app.domain.llm_summarization.services.llm_routines.build_map') as mock_build_map,
    patch('app.domain.llm_summarization.services.llm_routines.build_reduce') as mock_build_reduce:

    mock_build_map.return_value = mock_map_chain
    mock_build_reduce.return_value = mock_reduce_chain

    result = await summarize_article_text(
        llm=mock_llm,
        splitter=mock_splitter,
        title="Empty Article",
        url="https://example.com",
        content="",
        n_map_bullets=2
    )

    # Verify reduce chain was called with empty bullets
    mock_reduce_chain.invoke.assert_called_once()
    call_args = mock_reduce_chain.invoke.call_args[0][0]
    assert call_args["bullets"] == ""

    print("Empty content test passed!")

# Run the additional tests
await test_none_values()
await test_empty_content()

```

None values test passed!

Empty content test passed!